

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет программной инженерии и компьютерной техники

Дисциплина: Основы программной инженерии

Лабораторная работа №3

Выполнили студенты:

Егорова Варвара, Ташкинова Татьяна

Группа Р3223

Преподаватель:

Бострикова Дарья Константиновна

г. Санкт-Петербург

2025

Содержание:

Задание.....	3
Реализация:.....	5
Вывод:.....	13

Задание

Написать сценарий для утилиты Gradle, реализующий компиляцию, тестирование и упаковку в jar-архив кода проекта из лабораторной работы №3 по дисциплине "Веб-программирование".

Каждый этап должен быть выделен в отдельный блок сценария; все переменные и константы, используемые в сценарии, должны быть вынесены в отдельный файл параметров; MANIFEST.MF должен содержать информацию о версии и о запускаемом классе.

Сценарий должен реализовывать следующие цели (targets):

1. `compile` -- компиляция исходных кодов проекта.
2. `build` -- компиляция исходных кодов проекта и их упаковка в исполняемый jar-архив. Компиляцию исходных кодов реализовать посредством вызова цели `compile`.
3. `clean` -- удаление скомпилированных классов проекта и всех временных файлов (если они есть).
4. `test` -- запуск junit-тестов проекта. Перед запуском тестов необходимо осуществить сборку проекта (цель `build`).
5. `scp` - перемещение собранного проекта по scp на выбранный сервер по завершению сборки. Предварительно необходимо выполнить сборку проекта (цель `build`).
6. `xml` - валидация всех xml-файлов в проекте.
7. `doc` - добавление в MANIFEST.MF MD5 и SHA-1 файлов проекта, а также генерация и добавление в архив javadoc по всем классам проекта.
8. `native2ascii` - преобразование `native2ascii` для копий файлов локализации (для тестирования сценария все строковые параметры необходимо вынести из классов в файлы локализации).
9. `report` - в случае успешного прохождения тестов сохраняет отчет junit в формате xml, добавляет его в репозиторий git и выполняет commit.
10. `alt` - создаёт альтернативную версию программы с измененными именами переменных и классов (используя задание `replace/replaceregexp` в файлах параметров) и упаковывает её в jar-архив. Для создания jar-архива использует цель `build`.

11. diff - осуществляет проверку состояния рабочей копии, и, если изменения касаются классов, указанных в файле параметров выполняет commit в репозиторий svn.
12. history - если проект не удаётся скомпилировать (цель compile), загружается предыдущая версия из репозитория svn. Операция повторяется до тех пор, пока проект не удастся собрать, либо не будет получена самая первая ревизия из репозитория. Если такая ревизия найдена, то формируется файл, содержащий результат операции diff для всех файлов, изменённых в ревизии, следующей непосредственно за последней работающей.

Реализация:

```
import javax.xml.parsers.DocumentBuilderFactory
import org.xml.sax.SAXException
import java.nio.file.Files;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.security.MessageDigest

plugins {
    id 'java'
    id 'war'
    id 'org.hidetake.ssh' version '2.10.1'
}

group = 'org.example'
version = '1.0-SNAPSHOT'

repositories {
    mavenCentral()
}

java {
    sourceCompatibility = JavaVersion.VERSION_17
    targetCompatibility = JavaVersion.VERSION_17
}

dependencies {
    implementation("javax.ejb:javax.ejb-api:3.2.2")
    implementation("javax.servlet:javax.servlet-api:4.0.1")
    implementation("com.google.code.gson:gson:2.8.9")
    implementation("javax.enterprise:cdi-api:2.0-PFD2")
    implementation("org.primefaces:primefaces:10.0.0")
    implementation("org.postgresql:postgresql:42.6.1")

    implementation("javax.annotation:javax.annotation-api:1.3.2")
    implementation("org.hibernate:hibernate-core:5.6.3.Final")

    implementation("org.eclipse.persistence:eclipselink:3.1.0-M1")
    //    implementation("org.projectlombok:lombok:1.18.30")
    compileOnly('org.projectlombok:lombok:1.18.30')
    annotationProcessor('org.projectlombok:lombok:1.18.30')
    implementation("javax.faces:javax.faces-api:2.2-m12")
    implementation("javax.javaee-api:6.0")
}
```

```

testImplementation('org.junit.jupiter:junit-jupiter-api:5.9.3'
)

testRuntimeOnly('org.junit.jupiter:junit-jupiter-engine:5.9.3'
)
}

tasks.register("compile") {
    dependsOn(classes)
}

tasks.getByName("build").configure {
    dependsOn(tasks.getByName("war"))
}

tasks.getByName('test') {
    dependsOn(tasks.getByName('war'))
}

tasks.register('doc') {
    def manifestFile = file("$buildDir/tmp/MANIFEST.MF")
    manifestFile.parentFile.mkdirs()
    manifestFile.write("Manifest-Version: 1.0\n")

    String filePath = "$buildDir/tmp/MANIFEST.MF";

    MessageDigest shaDigest =
MessageDigest.getInstance("SHA-256")
    MessageDigest mdDigest = MessageDigest.getInstance("MD5")

    fileTree(dir: 'src/main/java', includes:
['**/*.java']).each { File file ->
        shaDigest.update(file.bytes)
        mdDigest.update(file.bytes)
    }
    String arr = "SHA-256: " +
shaDigest.digest().encodeHex().toString() + "\n" +
        "MD5: " + mdDigest.digest().encodeHex().toString()

    Files.write(Paths.get(filePath), arr.getBytes(),
StandardOpenOption.APPEND);

```

```

        println arr;
    }

tasks.getByName('doc').finalizedBy(tasks.getByName('javadoc'))

tasks.register('xml') {
    def projectRoot = project.projectDir
    inputs.dir(projectRoot)
    doLast {
        def factory = DocumentBuilderFactory.newInstance()
        def parser = factory.newDocumentBuilder()
        println "Validating all XML files in
${projectRoot}..."
        def invalidFiles = []
        fileTree(projectRoot).matching { include '**/*.xml'
    }.each { file ->
        def relativePath =
file.absolutePath.replace(projectRoot.absolutePath +
File.separator, "")
        try {
            parser.parse(file)
            println "Valid file: ${relativePath}"
        } catch (SAXException e) {
            println "Invalid file: ${relativePath} -
${e.message}"
            invalidFiles << file
        } catch (Exception e) {
            println "Error reading file: ${relativePath} -
${e.message}"
            invalidFiles << file
        }
    }
    if (!invalidFiles.isEmpty()) {
        throw new GradleException("XML validation failed
for ${invalidFiles.size()} file(s).")
    }
    println "All XML files are valid"
}
}

tasks.register('native2ascii') {
    doLast {

```

```

        def inputDir = file("src/main/resources")
        def outputDir = new
File("${project.buildDir}/asciiBuild")
        println "Converting .properties files from
${inputDir}"
        fileTree(inputDir).matching { include
'**/*.properties' }.each { inputFile ->
            def relativePath =
inputDir.toPath().relativize(inputFile.toPath()).toString()
            def outputFile = new File(outputDir, relativePath)
            outputFile.parentFile.mkdirs()
            outputFile.withWriter('ASCII') { writer ->
                inputFile.eachLine { line ->
                    def convertedLine = line.collect { ch ->
                        ch <= 127 ? ch :
String.format("\\u%04x", (int) ch)
                    }.join('')
                    writer.writeLine(convertedLine)
                }
            }
            println "${inputFile.name} -> ${outputFile.name}"
        }
        println "Output dir: ${outputDir}"
    }
}

```

```

tasks.register('report') {
    dependsOn(tasks.getByName('test'))
    println "Tests complited"
    doLast {
        def reportsDir = file("${buildDir}/test-results/test")
        def repoDir = file("${buildDir}/gitReports")
        fileTree(reportsDir).matching {
            include '**/*.java'
        }.each { file ->
            copy {
                from reportsDir
                into repoDir
                include '**/*.xml'
            }
            println "Copied"
            exec {
                commandLine 'git', 'add', file.name
            }
        }
    }
}

```



```

        }
    }
    exec {
        commandLine 'git', 'commit', '-m', 'Update JUnit
test reports'
    }

    println "JUnit reports saved to ${repoDir} and
committed to Git."
}

tasks.register('alt') {
    doLast {
        def sourceDir = file("src")

        fileTree(sourceDir).matching {
            include '**/*.java'

        }.each { file ->
            def text = file.text

            text = text.replaceAll('PointChecker',
'HitChecker')

            file.write(text)

            if (file.name.contains("PointChecker")) {
                def newFileName =
file.name.replace("PointChecker", "HitChecker")
                def newFile = new File(file.parentFile,
newFileName)
                file.renameTo(newFile)
            }
        }
        tasks.alt.finalizedBy(tasks.build.configure {
            tasks.'war'.archiveFileName = 'alt.war'
        })
    }
}

tasks.register('diff') {

```

```

doLast {
    def sourceDir = file("src")
    ByteArrayOutputStream svnStatus = new
ByteArrayOutputStream()
    exec {
        commandLine 'svn', 'status'
        standardOutput = svnStatus
    }
    String svnOutput = svnStatus.toString()
    fileTree(sourceDir).matching {
        include '**/*.java'
    }.each { file ->
        if (file.name in svnOutput) {
            exec {
                commandLine 'svn', 'add', file.name
            }
        }
    }
    exec {
        commandLine 'svn', 'commit', '-m', 'Добавлены'
    }
}
}

```

```

tasks.register("history") {
    group = "versioning"
    description = "Rollback until compileJava succeeds and
save diff with next revision (SVN)"
}

```

```

doLast {
    def runSvn = { args ->
        def output = new ByteArrayOutputStream()
        exec {
            commandLine 'svn', *args
            standardOutput = output
            ignoreExitValue = true
        }
        return output.toString().trim()
    }

    def getCurrentRevision = {
        def info = runSvn(['info'])
    }
}

```

```

        def revLine = info.readlines().find {
it.startsWith('Revision:') }
        if (revLine == null) throw new
GradleException("Can't determine current SVN revision.")
        return
Integer.parseInt(revLine.replace('Revision:', '').trim())
    }

    def currentRev = getCurrentRevision()
    def lastWorkingRev = null

    while (currentRev > 0) {
        println "Trying to compile at revision
$currentRev..."
        try {
            def result = exec {
                commandLine './gradlew', 'compile'
            }
            if (result.exitValue == 0) {
                println "Compilation succeeded at revision
$currentRev"
                lastWorkingRev = currentRev
                break
            }
        } catch (Exception e) {
            currentRev -= 1
            println "Compilation failed, updating to
revision $currentRev..."
            runSvn(['update', '-r',
currentRev.toString()])
        }

    }

    if (lastWorkingRev == null) {
        println "No working revision found!"
        return
    }

    def nextRev = lastWorkingRev + 1
    println "Generating diff between $lastWorkingRev and
$nextRev..."
    def diff = runSvn(['diff', '-r',
"$lastWorkingRev:$nextRev"])

```

```
        file("diff.txt").text = diff
        println "Diff saved to diff.txt."
    }
}

test {
    useJUnitPlatform()
    maxParallelForks = 1
    forkEvery = 0
    jvmArgs = ["-XX:MaxHeapSize=1G",
"-XX:MaxMetaspaceSize=128m"]
}
```

Вывод:

Во время выполнения лабораторной работы мы познакомились с утилитой для автоматизации процесса сборки Gradle, были написаны собственные сценарии сборки, а также модульные тесты на основе библиотеки JUnit для лабораторной работы по веб-программированию.